Understanding Your Data

Module 19 Study Notes • Initial Data Exploration

1. The Goal of Initial Exploration

Once you have successfully imported your dataset (via CSV, JSON, SQL, or Scraping), the very first step before building models or performing deep Exploratory Data Analysis (EDA) is to simply **understand what you have**. You achieve this by asking 8 fundamental questions about your data using Pandas functions.

2. The 8 Fundamental Questions

Q1: How big is the data?

You need to know the number of rows and columns to determine if memory management will be an issue.

> `df.shape`
>
> Returns a tuple representing *(rows, columns)*.

Q2: How does the data look?

You need a visual sense of the data values and formatting.

> `df.head()` : Shows the first 5 rows.
> `df.sample(5)` : Shows 5 **random** rows. This is often better than `head()` because it helps you avoid inherent biases (e.g., if the data is sorted, the first 5 rows might all look identical).

Q3: What are the Data Types of the columns?

Machine learning models require numerical data. You must identify which columns are strings (Objects) that need encoding, and which numerical columns are using excessive memory (e.g., storing a small integer as a float64).

```
df.info()
```

Provides a concise summary including column names, non-null counts, data types, and total memory usage.

Q4: Are there any missing values?

Missing data will crash ML algorithms. You need to know exactly how many missing values exist in each column to decide whether to drop them or fill them (Imputation).

```
df.isnull().sum()
```

Q5: How does the data look mathematically?

For numerical columns, you want a high-level statistical overview.

```
df.describe()
```

Calculates the count, mean, standard deviation, minimum, maximum, and percentiles (25%, 50%, 75%) for all numerical columns. This instantly helps you spot outliers or weird data entries.

Q6: Are there duplicate rows?

Duplicate data skews the training process and gives false confidence in the model's performance.

```
df.duplicated().sum()
```

Q7: How are columns correlated?

You need to know how the input columns relate to the output column (and to each other). If an input column has a correlation score of 0.0 with the target, it is useless and should be dropped.

```
df.corr()
```

Returns a Pearson correlation matrix ranging from -1 (strong inverse relationship) to 1 (strong direct relationship).